COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	September 4, 2026	**DATE SUBMITTED:**	September 7, 2026
NAME:	Mendoza, Nathaniel B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 7 **Sorting Algorithms Part 1**

1. PROCEDURE OUTPUT

Code + Console Screenshot

main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int SIZE = 100;

void displayArray(int arr[]);

int main() {

    // Create an array with unsorted list with 100 maximum size
    int unsortedArr[SIZE];

    // Use time(0) as seed
    std::srand(std::time(0));

    // Populate the array
    for (int i = 0; i < SIZE; i++) {
        unsortedArr[i] = std::rand() % 100 + 1;  // Random numbers between 1 and 100
    }
    std::cout << "Unsorted Array: " << std::endl;
    // Display the unsorted array
    displayArray(unsortedArr);
```

```
PS C:\Users\mendo\OneDrive\Documents\C++\MIDTERM\SortingAlgorithns> & C:\Users\mendo\
OneDrive\Documents\C++\MIDTERM\SortingAlgorithns\main.exe
Unsorted Array:
11 24 52 8 87 34 35 100 59 74 3 8 36 92 59 77 47 14 29 69 45 54 70 77 57 100 16 82 39
 81 84 13 4 80 94 49 91 59 91 90 10 50 93 24 97 12 73 14 2 54 78 27 5 92 8 53 6 68 99
 40 45 22 13 37 37 90 17 56 77 40 1 74 91 13 79 91 17 96 95 79 88 84 32 59 47 97 32 1
1 89 25 69 53 95 20 38 100 32 50 17 63
```

Observations	This program produces an array of 100 random integers that is generated.

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	sortingAlgo.h > insertionSort<T>(T [], size_t)
	1 #ifndef SORTINGALGO_H
	2 #define SORTINGALGO_H
	3
	4 #include <cstddef>
	5 #include <iostream>
	6 #include <utility>
	7
	8 template<typename T>
	9 void bubbleSort(T arr[], size_t arrSize) {
	10 for (size_t i = 0; i < arrSize; i++) {
	11 for (size_t j = i + 1; j < arrSize; j++) {
	12 if (arr[i] > arr[j]) {
	13 std::swap(arr[i], arr[j]);
	14 }
	15 }
	16 }
	17 }
	
	26 // Bubble sort the array
	27 bubbleSort(unsortedArr, SIZE);
	28 std::cout << "Sorted Array: " << std::endl;
	29 displayArray(unsortedArr);
	
	Sorted Array:
	1 2 3 4 5 6 8 8 8 10 11 11 12 13 13 13 14 14 16 17 17 17 20 22 24 24 25 27 29 32 32 3
	2 34 35 36 37 37 38 39 40 40 45 45 47 47 49 50 50 52 53 53 54 54 56 57 59 59 59 59 63
	68 69 69 70 73 74 74 77 77 77 78 79 79 80 81 82 84 84 87 88 89 90 90 91 91 91 91 92
	92 93 94 95 95 96 97 97 99 100 100 100

Observations	Bubble Sort repeatedly compares and swaps adjacent elements depending on whether it's higher to the next index value, making it the slowest of the algorithms tested. **Best Case Time Complexity**: $O(N^2)$ because it needs to check every index or O(N), if the array is already sorted. **Worst Case Time Complexity**: $O(N^2)$ because it needs to do comparisons and swapping.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	

```cpp
template<typename T>
int routineSmallest(T arr[], int K, const int arrSize) {
    int position = K;
    T smallestElem = arr[K];

    for (int j = K + 1; j < arrSize; j++) {
        if (arr[j] < smallestElem) {
            smallestElem = arr[j];
            position = j;
        }
    }
    return position;
}

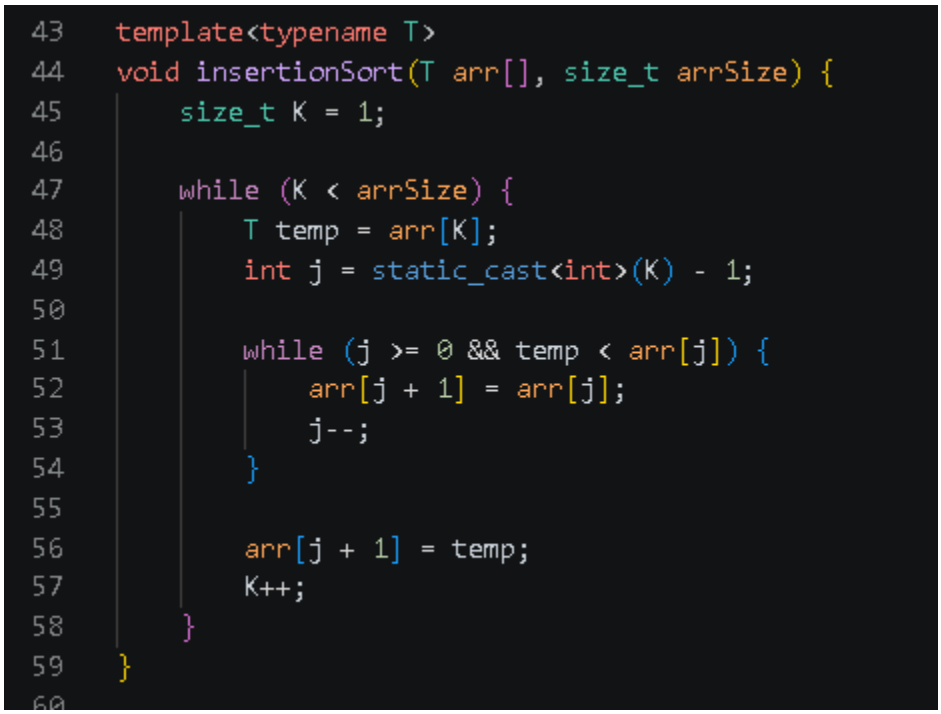
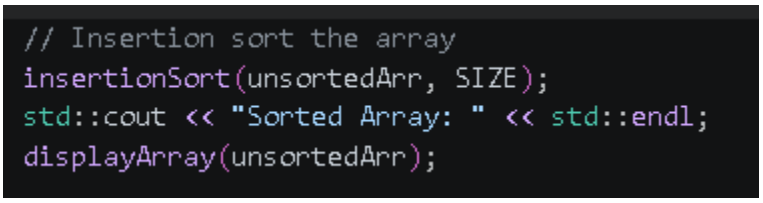
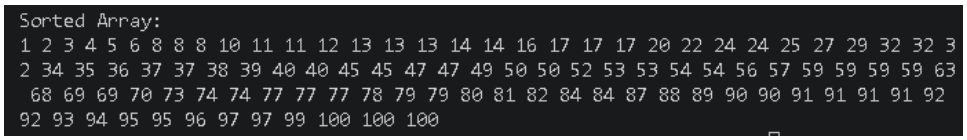
template<typename T>
void selectionSort(T arr[], size_t arrSize) {
    for (size_t i = 0; i < arrSize; i++) {
        int POS = routineSmallest(arr, i, static_cast<int>(arrSize));
        T temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;
    }
}
```

```cpp
// Selection sort the array
selectionSort(unsortedArr, SIZE);
std::cout << "Sorted Array: " << std::endl;
displayArray(unsortedArr);
```

```
Sorted Array:
1 2 3 4 5 6 8 8 8 10 11 11 12 13 13 13 14 14 16 17 17 17 20 22 24 24 25 27 29 32 32 3
2 34 35 36 37 37 38 39 40 40 45 45 47 47 49 50 50 52 53 53 54 54 56 57 59 59 59 59 63
 68 69 69 70 73 74 74 77 77 77 78 79 79 80 81 82 84 84 87 88 89 90 90 91 91 91 91 92
92 93 94 95 95 96 97 97 99 100 100 100
```

Observations	Selection Sort looks for the smallest remaining element and swaps it into place. **Best Case Time Complexity**: $O(N^2)$ happens when the array is already sorted it still requires to scan the whole array . **Worst Case Time Complexity**: $O(N^2)$ happens in any situation as it always executes comparisons and swapping a different way.

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	  
Observations	Insertion Sort shifts larger elements right to insert each item into its correct spot, performing fastest on small datasets. **Best Case Time Complexity**: O(N) if only the array is already sorted and it will only do one comparison without swapping/shifting. **Worst Case Time Complexity**: O(N^2) happens in any situation as it always performs comparisons and shifting all array index values.

```
template<typename T>
void insertionSort(T arr[], size_t arrSize) {
    size_t K = 1;

    while (K < arrSize) {
        T temp = arr[K];
        int j = static_cast<int>(K) - 1;

        while (j >= 0 && temp < arr[j]) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = temp;
        K++;
    }
}
```

```
// Insertion sort the array
insertionSort(unsortedArr, SIZE);
std::cout << "Sorted Array: " << std::endl;
displayArray(unsortedArr);
```

```
Sorted Array:
1 2 3 4 5 6 8 8 8 10 11 11 12 13 13 13 14 14 16 17 17 17 20 22 24 24 25 27 29 32 32 3
2 34 35 36 37 37 38 39 40 40 45 45 47 47 49 50 50 52 53 53 54 54 56 57 59 59 59 59 63
 68 69 69 70 73 74 74 77 77 77 78 79 79 80 81 82 84 84 87 88 89 90 90 91 91 91 91 92
92 93 94 95 95 96 97 97 99 100 100 100
```

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PSEUDOCODE of Algorithm

```
START
   CREATE Array "Candidates" = ["Name1" , “Name2”, “Name3”,  “Name4”, “Name5”]
   SET Array maxSize = 100
   CREATE Array votes[maxSize]
   CREATE Array voteCount[5] = {}

// Populate the array with random votes (1 to 5)
   FOR i = 0 TO maxSize - 1 DO
      votes[i] = randNumber(1, 5)
   END FOR

// Calling the best Sorting Algorithm (insertionSort)
   CALL insertionSort(votes, SIZE)

// Count votes per candidate
   FOR i = 0 TO maxSize - 1 DO
      voteCounts[votes[i] - 1] = voteCounts[votes[i] - 1] + 1
   END FOR

// Find the winner
   maxVotes = 0
   winnerIndex = 0
   FOR i = 0 TO 4 DO
      IF voteCounts[i] > maxVotes THEN
         maxVotes = voteCounts[i]
         winnerIndex = i
      END IF
   END FOR

   PRINT vote counts per candidate
   PRINT winner candidate name and total votes
END
```

Explanation: First before anything else is I will create an array for the Candidates as string and list their names. Then, I'll create a maxSize of 100 and implement that to another array called votes. Additionally, since there's 5 listed candidates, I will set all of its index value to none or all of it is 0. After that, I need to populate the array where I need to input randomNumber between 1-5 among all the 100 indexes from voting and end that loop. Then from my sortingAlgo header file, I will only call the insertionSort function as it is the once that will be utilized and its the fastest (*Proof at justification*). Then after its sorted, I will also apply another linear sorting via insertSort so that O(N) will be applied. Basically, my goal is to go from $O(N^2)$ by sorting then apply O(N) to the sorted array for the best time complexity. Lastly, it will find the winner by doing a comparison on how much that index value is gotten and counts it to that candidate and prints the winner and its total votes among all.

SCREENSHOT of Algorithm Code

supplementaryMain.cpp > ...

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <string>
#include "sortingAlgo.h"

const int SIZE = 100;

int main() {
    std::string candidates[5] = {
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustin",
        "Deja Jayla Banaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

    int votes[SIZE];
    int voteCounts[5] = {0};

    // Seed random generator
    std::srand(static_cast<unsigned int>(std::time(0)));

    // Generate random votes between 1 and 5
    for (int i = 0; i < SIZE; i++) {
        votes[i] = std::rand() % 5 + 1;
    }

    std::cout << "--- UNSORTED VOTES ARRAY ---\n";
    for (int i = 0; i < SIZE; i++) {
        std::cout << votes[i] << " ";
    }
    std::cout << "\n\n";

    // Sort the vote array using Insertion Sort
    insertionSort(votes, SIZE);

    std::cout << "--- SORTED VOTES ARRAY ---\n";
    for (int i = 0; i < SIZE; i++) {
        std::cout << votes[i] << " ";
    }
    std::cout << "\n\n";
```

```
    // Vote Counting
    for (int i = 0; i < SIZE; i++) {
        voteCounts[votes[i] - 1]++;
    }

    // Finding WInner
    int maxVotes = -1;
    int winnerIndex = 0;

    std::cout << "ELECTION RESULTS: \n";
    for (int i = 0; i < 5; i++) {
        std::cout << "Candidate " << (i + 1) << " (" << candidates[i] << "): "
                  << voteCounts[i] << " votes\n";
        if (voteCounts[i] > maxVotes) {
            maxVotes = voteCounts[i];
            winnerIndex = i;
        }
    }

    std::cout << "\nWINNER: " << candidates[winnerIndex]
              << " with " << maxVotes << " votes!\n";

    return 0;
}
```

Analysis: First is the included libraries required for the main program. Then, setted a maxSize of 100 and then starts the main program. Where it has an array string of all Candidates, and then another array for votes then voteCounts array as 0. Then for the votes array it has a total index of 100 then I applied random numbers between 1 to 5 index value as minimum and max to fill all those indexes of 100 from votes. Then, it is sorted bia calling the insertionSort array with time complexity of $O(N^2)$ and then after that prints the unsorted then sorts it again so that it can be approached via O(N) by the insertion sort. Then, takes all the index values and count them how much. Lastly, it finds the winning candidate and prints the winner and all of the number of votes among all candidates.

OUTPUT TESTING

```
--- UNSORTED VOTES ARRAY ---
1 5 4 3 5 2 3 2 2 2 3 2 1 2 1 1 1 2 3 2 3 4 4 3 4 5 3 5 1 2 2 3 3 4 5 4 3 5 1 1 5 2 1 4 1 4 1 5 1 5 5 2 2 3 3 4 3 3 1 1 3
1 5 2 2 5 1 4 5 1 4 5 3 5 3 4 4 1 3 3 2 4 2 3 4 1 4 2 3 4 5 1 5 3 5 2 2 4 3 5

--- SORTED VOTES ARRAY ---
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

ELECTION RESULTS:
Candidate 1 (Bo Dalton Capistrano): 20 votes
Candidate 2 (Cornelius Raymon Agustin): 20 votes
Candidate 3 (Deja Jayla Banaga): 23 votes
Candidate 4 (Lalla Brielle Yabut): 18 votes
Candidate 5 (Franklin Relano Castro): 19 votes

WINNER: Deja Jayla Banaga with 23 votes!
PS C:\Users\mendo\OneDrive\Documents\C++\MIDTERM\SortingAlgorithms>
```

Observation: This shows the Unsorted array before getting sorted via Insertion Sort and then takes the winning candidate and its total votes among all other candidates.

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1	20	20
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2	20	20
3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3	23	23
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4	18	18
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5	19	19

Question: *Was your developed vote counting algorithm effective? Why or why not?*

Yes, the algorithm was effective for me. Because, sorting an unsorted array and taking the identical index values blocks a straightforward counting and boundary verification. By grouping identical candidate IDs together, the algorithm eliminates the need to perform arbitrary lookups across an unsorted dataset. So going from $O(N^2)$ time complexity before performing the O(N) counting step, it successfully organizes the data and accurately determines the winning candidate.

JUSTIFICATION

```
----------------------------------------
1. BUBBLE SORT RESULT:
Candidate 1 (Bo Dalton Capistrano): 19 votes
Candidate 2 (Cornelius Raymon Agustin): 18 votes
Candidate 3 (Deja Jayla Banaga): 22 votes
Candidate 4 (Lalla Brielle Yabut): 19 votes
Candidate 5 (Franklin Relano Castro): 22 votes
-> WINNER: Deja Jayla Banaga with 22 votes!
   Execution Time: 23 microseconds
----------------------------------------
2. SELECTION SORT RESULT:
Candidate 1 (Bo Dalton Capistrano): 19 votes
Candidate 2 (Cornelius Raymon Agustin): 18 votes
Candidate 3 (Deja Jayla Banaga): 22 votes
Candidate 4 (Lalla Brielle Yabut): 19 votes
Candidate 5 (Franklin Relano Castro): 22 votes
-> WINNER: Deja Jayla Banaga with 22 votes!
   Execution Time: 31 microseconds
----------------------------------------
3. INSERTION SORT RESULT:
Candidate 1 (Bo Dalton Capistrano): 19 votes
Candidate 2 (Cornelius Raymon Agustin): 18 votes
Candidate 3 (Deja Jayla Banaga): 22 votes
Candidate 4 (Lalla Brielle Yabut): 19 votes
Candidate 5 (Franklin Relano Castro): 22 votes
-> WINNER: Deja Jayla Banaga with 22 votes!
   Execution Time: 16 microseconds
========================================
```

All three sorting algorithms were applied to evaluate its speed based on time complexity, and Insertion Sort proved to be the most efficient with an execution time of 16 microseconds (compared to 23 microseconds for Bubble Sort and 31 microseconds for Selection Sort).

The results prove that while sorting introduces an initial $O(N^2)$ worst-case time complexity., Insertion Sort handles this phase with lesser comparisons on small datasets which shows lesser usage of time, just by quickly organizing the vote IDs. Linear O(N) counting passes can take place without struggle reading the sorted array..

CONCLUSION

Overall, this activity helped me clearly understand the practical differences between sorting algorithms and enhanced my ability to evaluate and select the best computational approach for a given problem. Additionally, I learned how to accurately measure execution time at the code level using the <chrono> library, and I relearned how essential and important writing a clear pseudocode is for structuring and translating complex logic into working code.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Google Search (Gemini)

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Finding a way for Comparison of Time complexity between available/taught sorting

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Asked what library to include & code implementation in terms of finding time complexity.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.